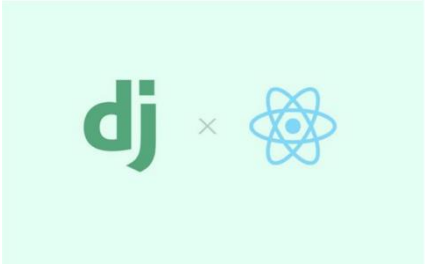
# Desarrollo de Aplicaciones Empresariales

Tema de la Sesión 07:  ORM de Django.

C24 – Diseño y Desarrollo de Software



Tecsup

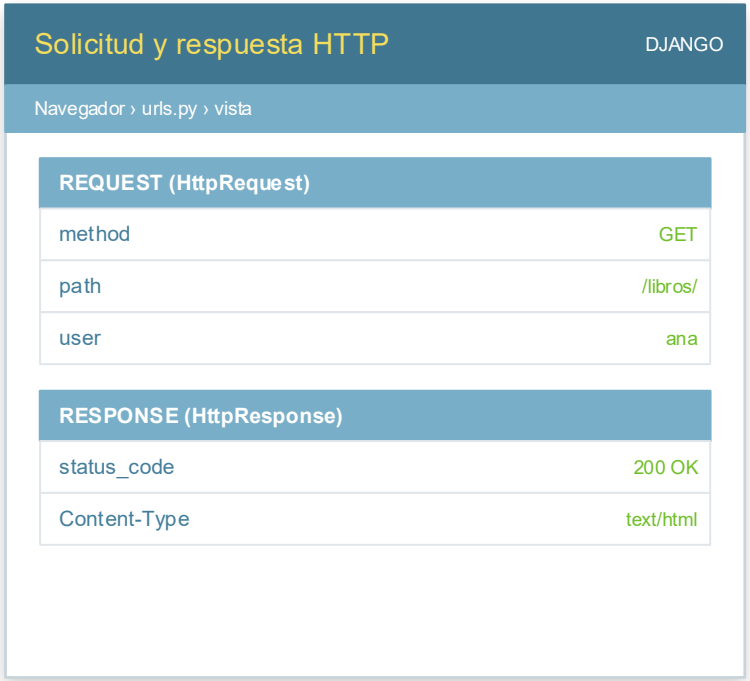
# Recordamos

# ¿Qué es una vista?

Es una **función de Python** que recibe una solicitud web y devuelve una respuesta.

Django crea un objeto HttpRequest con los datos de la solicitud y se lo entrega a la vista como primer parámetro.

Se escribe en `views.py` y se conecta a una URL en urls.py.




Tecsup

# Características del motor de plantillas



### Variables

Muestran datos del contexto con {{ }}.



### Autoescape

Escapa el HTML para prevenir ataques XSS.



### Herencia

Las páginas reutilizan una plantilla base.



### Filtros

Transforman valores: fechas, mayúsculas, textos.



### Etiquetas

Añaden lógica con {% %}: bucles y condiciones.



### Inclusión

Reutiliza fragmentos con {% include %}.

Tecsup

# Ciclo solicitud – respuesta

1. **El navegador solicita**

```
→ GET http://127.0.0.1:8000/libros/
```

2. **urls.py enruta**

```
→ path('libros/', views.lista_libros)
```

3. **La vista procesa**

```
→ libros = Libro.objects.all()
```

4. **Se renderiza**

```
→ render(request, 'lista.html', ctx)
```

*La vista siempre devuelve un objeto HttpResponse, que Django envía de vuelta al navegador.*

Tecsup



# Conocimientos previos

Cuestionario interactivo (Kahoot!)

# Resultados

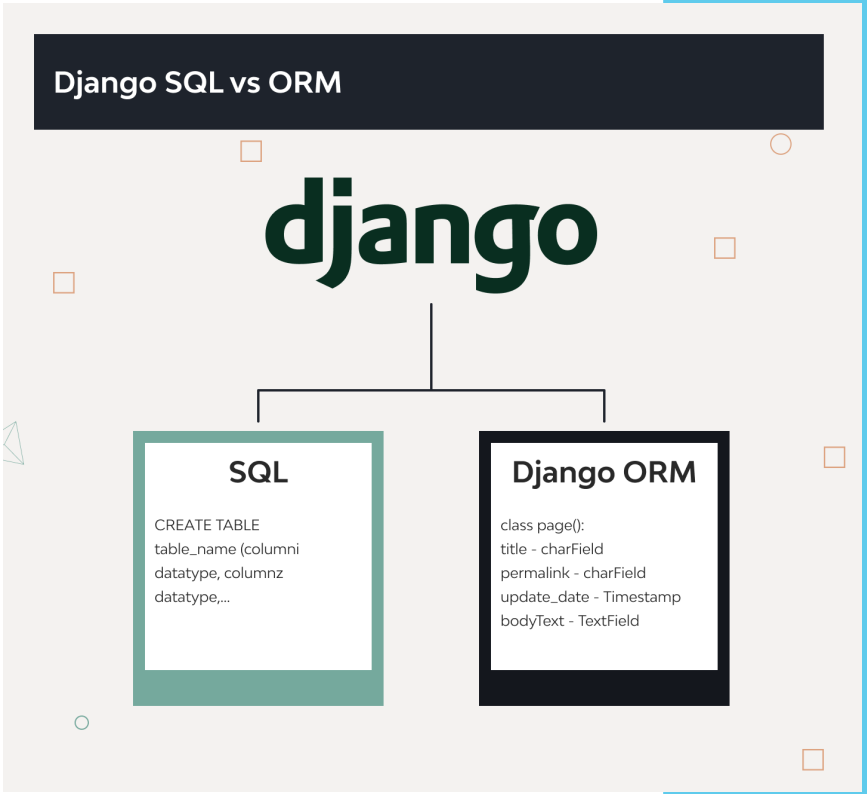
# Motivación

Ya registraste 500 libros desde el panel de administración.

Si una aplicación web necesita registrar, consultar, modificar y eliminar miles de usuarios o productos almacenados en una base de datos,

**¿sería necesario escribir manualmente una consulta SQL para cada operación?**

**¿Cómo podría Django facilitar este trabajo mediante su ORM?**




Tecsup

# Objetivos



Identificar las características del módulo ORM incorporado en Django.

Al finalizar la sesión podrás definir modelos, aplicar migraciones y crear, consultar, actualizar y eliminar registros con el ORM de Django.

Tecsup

# Tema Semana 07

# ORM de Django

- Modelos y migraciones: de clases de Python a tablas.
- QuerySets: consultas, relaciones y operaciones CRUD sin escribir SQL.

Tecsup

# Actividad 01: Individual

# Usando tu IA favorita

Individual – Haciendo uso de ChatGPT investiga: qué es un ORM, qué ventajas y desventajas tiene frente a escribir SQL directamente y qué ORM usan otros frameworks (por ejemplo, Eloquent en Laravel).

Contrasta la respuesta de la IA con la documentación oficial de Django



# Presentamos resultados

# ¿Qué es un ORM?

ORM (Object-Relational Mapping) conecta los objetos de Python con las tablas de una base de datos relacional.

Cada clase del modelo es una tabla, cada atributo una columna y cada objeto una fila.

Django trae su propio ORM: no necesitas instalar nada adicional.




Tecsup

# Características del ORM de Django



## Modelos

Cada clase de Python representa una tabla.



## Seguridad

Usa consultas parametrizadas contra la inyección SQL.



## Relaciones

ForeignKey, ManyToMany y OneToOne.



## QuerySets

Consultas encadenables que se ejecutan al usarse.



## Migraciones

Llevan los cambios del modelo a la base de datos.



## Multimotor

SQLite, PostgreSQL, MySQL, MariaDB y Oracle.

Tecsup

# Configuración de la base de datos

Django se conecta a la base de datos según el ajuste DATABASES; por defecto usa SQLite:

```python
# settings.py (desarrollo)
DATABASES = {
  'default': {
    'ENGINE':
      'django.db.backends.sqlite3',
    'NAME': BASE_DIR / 'db.sqlite3',
  }
}
```

```python
# settings.py (producción)
DATABASES = {
  'default': {
    'ENGINE':
      'django.db.backends.postgresql',
    'NAME': 'biblioteca',
    'USER': 'admin',
    'HOST': 'localhost',
    'PORT': '5432',
  }
}
```

*Para PostgreSQL instala el conector (pip install psycopg); los modelos y las consultas no cambian.*

Tecsup

# Del modelo a la base de datos

1. **Defines el modelo**

```
→ class Libro(models.Model)
```

2. **Creas la migración**

```
→ python manage.py makemigrations
```

3. **Aplicas la migración**

```
→ python manage.py migrate
```

4. **Consultas con el ORM**

```
→ Libro.objects.filter(anio__gt=1960)
```

*Cada vez que cambies un modelo, repite makemigrations y migrate para actualizar las tablas.*

Tecsup



Es tiempo de
**PREGUNTAS**

Tecsup

# Definir un modelo

- Un modelo es una clase que hereda de models.Model.
- Cada atributo es un campo y define el tipo de columna.
- Django agrega automáticamente un campo id como clave primaria.

```python
# biblioteca/models.py
from django.db import models

class Autor(models.Model):
    nombre = models.CharField(max_length=100)
    pais = models.CharField(max_length=50)

class Libro(models.Model):
    titulo = models.CharField(max_length=200)
    anio = models.IntegerField()
    autor = models.ForeignKey(Autor,
        on_delete=models.CASCADE)
```

Tecsup

# Del ORM al SQL

Cada operación del ORM se traduce a SQL; puedes ver la consulta con print(queryset.query).

```
# Crear un registro
Libro.objects.create(
    titulo='Rayuela', anio=1963,
    autor=cortazar)

# Consultar registros
Libro.objects.filter(
    anio__lt=1970)
```

**SQL generado (crear)**

```
INSERT INTO biblioteca_libro
(titulo, anio, autor_id) VALUES (...)
```

**SQL generado (consultar)**

```
SELECT * FROM biblioteca_libro
WHERE anio < 1970
```

Tecsup

# Consultas con QuerySets

objects es el Manager del modelo: todas las consultas empiezan ahí.

Un QuerySet es perezoso: la consulta se ejecuta recién cuando lo recorres o lo conviertes en lista.

```
# Consultas frecuentes
Libro.objects.all()
Libro.objects.get(id=1)
Libro.objects.filter(anio__gte=1960)
Libro.objects.exclude(anio__gte=1960)
Libro.objects.order_by('-anio')[:5]

# Se pueden encadenar
Libro.objects.filter(anio__gte=1960) \
    .exclude(autor__pais='Perú') \
    .order_by('titulo')
```

Tecsup

# Métodos y lookups más usados

| Método / lookup | Función |
|---|---|
| `filter()` | Devuelve los registros que cumplen la condición |
| `exclude()` | Devuelve los registros que no la cumplen |
| `get()` | Devuelve un solo registro (error si no existe o hay varios) |
| `order_by()` | Ordena: 'campo' ascendente, '-campo' descendente |
| `__icontains` | Contiene un texto, sin distinguir mayúsculas |
| `__gte / __lte` | Mayor o igual / menor o igual que un valor |
| `__in` | El valor está dentro de una lista |
| `count() / exists()` | Cuenta los resultados o verifica si hay alguno |

Tecsup



Es tiempo de
**PREGUNTAS**

Tecsup

# Tipos de relaciones

Las relaciones entre tablas se declaran como campos del modelo:

### ForeignKey

Uno a muchos: un autor escribe muchos libros.

```
ForeignKey(Autor)
```

### ManyToManyField

Muchos a muchos: un libro tiene varios géneros.

```
ManyToManyField(Genero)
```

### OneToOneField

Uno a uno: cada usuario tiene un solo perfil.

```
OneToOneField(User)
```

`on_delete: models.CASCADE` → al borrar un autor, también se borran sus libros

Tecsup

# Consultas entre relaciones

```
# Del libro al autor
libro = Libro.objects.get(id=1)
libro.autor.nombre

# Del autor a sus libros
autor = Autor.objects.get(id=2)
autor.libro_set.all()

# Filtrar por un campo relacionado
Libro.objects.filter(autor__pais='Perú')
```

- Accede al objeto relacionado con un punto: libro.autor.
- En sentido inverso usa <modelo>_set: autor.libro_set.all().
- Usa doble guion bajo para filtrar por otra tabla: autor__pais.

Tecsup

# Operaciones CRUD con el ORM

Con el ORM creas, lees, actualizas y eliminas registros sin escribir SQL:

| CREATE | READ | UPDATE | DELETE |
| --- | --- | --- | --- |
| create() / save() | all() / filter() / get() | save() / update() | delete() |



`libro.save()` Guarda los cambios de un solo registro.



`filter(...).update()` Actualiza varios registros en una sola consulta.



`libro.delete()` Elimina el registro (y sus dependientes con CASCADE).

Tecsup



Es tiempo de
**PREGUNTAS**

Tecsup

# Ventajas del ORM



## Productividad

Escribes Python en lugar de SQL y obtienes objetos listos para usar.



## Seguridad

Las consultas son parametrizadas: el ORM evita la inyección SQL.



## Portabilidad

Cambias de SQLite a PostgreSQL o MySQL sin reescribir las consultas.

Tecsup

# Buenas prácticas



Define __str__ en cada modelo para identificar los registros en el admin y en la consola.



Evita el problema N+1: usa select_related() y prefetch_related() al consultar relaciones.



Versiona las migraciones con Git y no modifiques las tablas a mano.



En las vistas, usa get_object_or_404() para manejar registros que no existen.

Tecsup

# Actividad 02: Grupal

- Cada equipo diseña los modelos (models.py) y escribe las consultas del ORM que resuelven los requerimientos del caso.
- Exponen (2 min. por equipo) y se contrasta con la propuesta de solución presentada por el docente.

Tecsup

# Caso de estudio

- Sistema de pedidos para Ferretería Andina S.A.C.
- Aplicamos modelos, relaciones y consultas del ORM a un caso empresarial.



Tecsup

# Caso: Ferretería Andina S.A.C.

## Contexto

El portal web ya muestra el catálogo de 1 200 productos.

Los pedidos de los clientes empresariales llegan por correo y se registran en Excel.

**Gerencia solicita guardarlos en la base de datos y consultarlos desde Django.**

- **R1** Registrar clientes (RUC y razón social) y productos (precio, stock y estado activo).
- **R2** Un pedido pertenece a un cliente e incluye varios productos con su cantidad.
- **R3** Listar los productos con stock menor a 10, de menor a mayor stock.
- **R4** Mostrar los pedidos de un cliente por su RUC, del más reciente al más antiguo.
- **R5** Desactivar en una sola consulta todos los productos sin stock.

Tecsup

# Caso: ¿qué debe resolver tu equipo?

En equipos, analicen los requerimientos y respondan (15 min):

### 1 Modelos

- ¿Qué modelos y campos necesitas? (R1)
- ¿Qué campo identifica a cada cliente?

### 2 Relaciones

- ¿Qué relación une Pedido con Cliente? (R2)
- ¿Dónde guardas la cantidad de cada producto? (R2)

### 3 Consultas

- ¿Qué lookups resuelven R3 y R4?
- ¿Cómo actualizas varios productos a la vez? (R5)

*Entregable: esquema de models.py y consultas del ORM para R3, R4 y R5; lo exponen en 2 minutos por equipo.*

https://miro.com/app/board/uXjVHgI2R78=/?share_link_id=39283057558

Tecsup

# Presentamos la propuesta

- Sistema de pedidos para Ferretería Andina S.A.C.
- Aplicamos modelos, relaciones y consultas del ORM a un caso empresarial.

Tecsup



Es tiempo de
**PREGUNTAS**

Tecsup



# ¿Qué aprendimos hoy?

Cuestionario interactivo (Kahoot!)

# Conclusiones

- El ORM permite trabajar con la base de datos usando clases y objetos de Python en lugar de SQL.
- Cada modelo es una tabla, y las migraciones mantienen el esquema sincronizado con el código.
- Con los QuerySets se crean, consultan, actualizan y eliminan registros, incluso a través de relaciones.
- ForeignKey, ManyToManyField y OneToOneField representan las relaciones entre tablas.

Tecsup

# Material Complementario

Tutorial de Django Parte 3: uso de modelos (MDN)

- https://developer.mozilla.org/es/docs/Learn/Server-side/Django/Models

Documentación oficial de Django: Making queries

- https://docs.djangoproject.com/en/stable/topics/db/queries/

Tecsup

# La próxima sesión veremos

- Django REST framework: introducción a servicios web, métodos y estados HTTP, REST, RESTful y API.
- Práctica calificada N.° 2.

Tecsup

Tecsup®

TECNOLOGÍA
CON SENTIDO